Q.2] Predict the output:

```
psvm(){
    int n = 2;
    P(n);
}
void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}

void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}

void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}

void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}

void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}

void P(int n){
    if(n==0){
        return;
    }
    Print("X");
    P(n-1);
    Print("Y");
    P(n-1);
}
```

Condition satisfied:

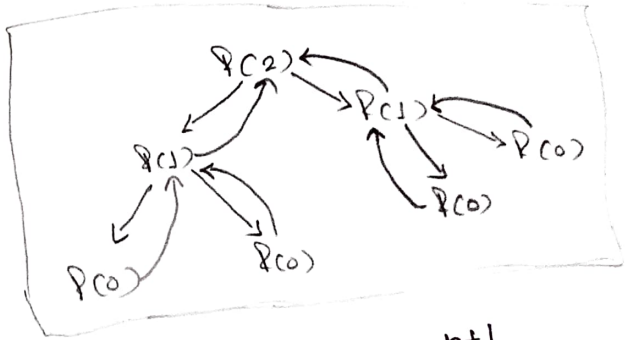
Condition Satisfied

Condition Satisfied

Condition Satisfied

Output

X
X
Y
Y
X
Y



Total Function Calls $= 2^{n+1} - 1$

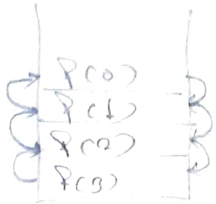
$= 2^3 - 1$

$= 8 - 1$

$\boxed{= 7}$

Q.5] Predict the output

```
psvm(){
    int n = 3;
    fun(n);
}
void fun(int n){
    if(n==0){
        return;
    }
    print(n+" ");
    f(n-1);
    print((n+1)+" ");
}
```



#output:
3
2
1
2
3
4

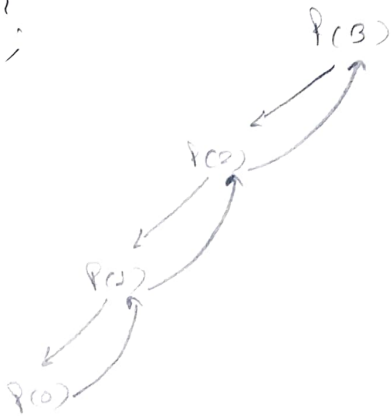
- Return
- Call

```
void fun(int n){
    if(n==0){
        return;
    }
    print(n+" ");
    f(n-1);
    print((n+1)+" ");
}
```

```
void fun(int n){
    if(n==0){
        return;
    }
    print(n+" ");
    f(n-1);
    print((n+1)+" ");
}
```

```
void fun(int n){
    if(n==0){
        return;
    }
    print(n+" ");
    f(n-1);
    print((n+1)+" ");
}
```

Recursive Tree



f(3) → 1
f(2) → 1
f(1) → 1
f(0) → 1

$\boxed{\text{Total calls} = 4}$

Q.2] Predict the output:

```
void main(){
    int n = 2;
    Fun(n);
}
void Fun(int n){
    if(n==0){
        return;
    }
    Print("x");
    Fun(n-1);
    Print("y");
    Fun(n-1);
}
```

```
void Fun(int n){
    if(n==0){
        return;
    }
    Print("x");
    F(n-1);
    Print("y");
    F(n-1);
}
```

```
void Fun(int n){
    if(n==0){
        return;
    }
}
```

Condition satisfied!

```
void Fun(int n){
    if(n==0){
        return;
    }
}
```

Condition satisfied

```
void F(int n){
    if(n==0){
        return;
    }
    Print("x");
    F(n-1);
    Print("y");
    Fun(n-1);
}
```

```
void F(int n){
    if(n==0){
        return;
    }
}
```

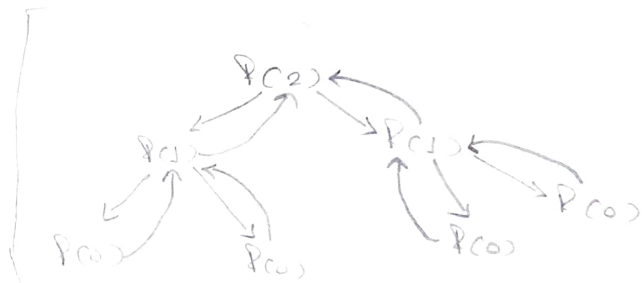
Condition satisfied

```
void F(int n){
    if(n==0){
        return;
    }
}
```

Condition satisfied

output

x
x
y
y
x
y



Q3) Predict the output:

```
psvm(){
    int n=4;
    Run(n);
}
void Run(int n){
    if( n <=1){
        Print(n);
        return;
    }
    Run(n-2);
    Print(n);
    Run(n-2);
}
```

Output

0
2
0
4
0
2
0

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
    Run(n-2);
    Print(n);
    Run(n-2);
}
```

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
    Run(n-2);
    Print(n);
    f(n-2);
}
```

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
}
```

Condition Satisfied

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
}
```

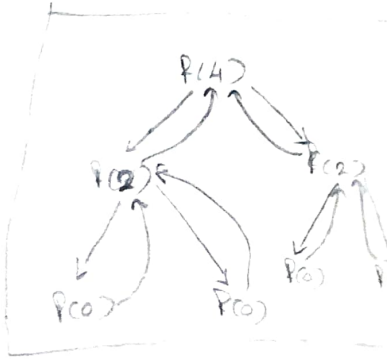
Condition Satisfied

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
}
```

Condition Satisfied

```
void Run(int n){
    if(n<=1){
        Print(n);
        return;
    }
}
```

Condition Satisfied



Total call = 7

Q.4) Predict the output:

```
psvm() {
    int n = 2;
    Run(n);
}
```

Output

1
4
1

```
void Run(int n) {
    if (n == 0) {
        return;
    }
    Run(n-1);
    Print(n*n);
    Run(n-1);
}
```

2

```
void Run(int n) {
    if (n == 0) {
        return;
    }
    Run(n-1);
    Print(n*n);
    Run(n-1);
}
```

1

```
void Run(int n) {
    if (n == 0) {
        return;
    }
    Run(n-1);
    Print(n*n);
    Run(n-1);
}
```

0

```
void Run(int n) {
    if (n == 0) {
        return;
    }
    Run(n-1);
    Print(n*n);
    Run(n-1);
}
```

0

```
void Run(int n) {
    if (n == 0) {
        return;
    }
}
```

Condition Satisfied

0

```
void Run(int n) {
    if (n == 0) {
        return;
    }
}
```

Condition Satisfied

0

```
void Run(int n) {
    if (n == 0) {
        return;
    }
}
```

Condition Satisfied

$= 2^{n+1} - 1$

$= 2^3 - 1$

$\boxed{= 7}$ Total Function call = 7

Q = Predict the output

```
void main() {
    int n = 5;
    Fun(n);
}

void Fun(int n) {
    if (n <= 0) {
        return;
    }
    Print(n);
    F(n-2);
    Print(n);
}
```

```
void Fun(int n) {
    if (n <= 0) {
        return;
    }
    Print(n);
    F(n-2);
    Print(n);
}
```

```
void Fun(int n) {
    if (n <= 0) {
        return;
    }
    Print(n);
    F(n-2);
    Print(n);
}
```

```
void Fun(int n) {
    if (n <= 0) {
        return;
    }
    Print(n);
    F(n-2);
    Print(n);
}
```

Condition satisfied

Output

5
3
1
1
3
5

Total Function call = 4

Q.6) Predict the output:

```
void main() {
    int n = 8;
    Run(n);
}

void Run (int n) {
    if (n<=1) {
        Print (n);
        return;
    }
    Print (n);
    Run(n/2);
    Print(n);
}
```

Output

8
4
2
1
2
4
8

```
void Run (int n) {
    if (n<=1) {
        Print (n);
        return;
    }
    Print (n);
    Run(n/2);
    Print(n);
}
```

```
void Run (int n) {
    if (n<=1) {
        Print (n);
        return;
    }
    Print (n);
    Run(n/2);
    Print (n);
}
```

```
void Run (int n) {
    if (n<=1) {
        Print (n);
        return;
    }
    ~~Print~~
}
```

condition satisfied

Total function call = 4

Q.17) Predict the output:

```
psvm(){
    int n = 3;
    Run(n);
}
```

Output

```
A 3
A 2
A 1
B 2
B 4
B 6
```

5

```
void Run(int n){
    if(n<=0){
        return;
    }
    Print("A"+n);
    Run(n-1);
    Print("B"+(n*2));
}
```

2

```
void Run(int n){
    if(n<=0){
        return;
    }
    Print("A"+n);
    Run(n-1);
    Print("B"+(n*2));
}
```

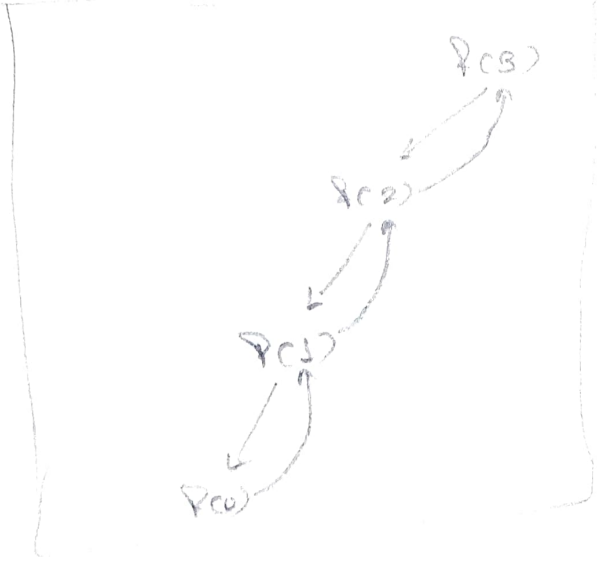
1

```
void Run(int n){
    if(n<=0){
        return;
    }
    Print("A"+n);
    Run(n-1);
    Print("B"+(n*2));
}
```

0

```
void Run(int n){
    if(n<=0){
        return;
    }
}
```

Condition satisfied



Total Function calls = 4

Q.8) Predict the output:

```
psvm(){
    int n = 4;
    Fun(n);
}
```

Output

1
2
3
-1
-2
0

```
void Fun(int n){ // 4
    if (n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 3
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 2
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 1
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 0
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // -1
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // 2
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 1
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 0
    if(n<=0){ } — Condition satisfy
    return;
}
```

```
void Fun(int n){ // -1
    if(n<=0){ } — Condition satisfy
    return;
}
```

```
void Fun(int n){ // 0
    if(n<=0){ }
    return;
}
```

```
void Fun(int n){ // 1
    if(n<=0){
        return;
    }
    Fun(n-1);
    Print(n);
    Fun(n-2);
}
```

```
void Fun(int n){ // 0
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // -1
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // 0
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // 0
    if(n<=0){ } — condition satisfied
    return;
}
```

```
void Fun(int n){ // -1
    if(n<=0){ } — condition satisfied
    return;
}
```

Q.3) Predict the output:

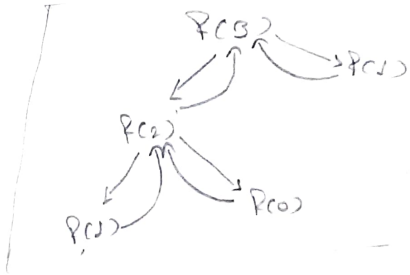
```
Pvoid(){
    int n=3;
    Pun(n);
}

void Pun(int n){
    if (n<=1){
        Point(n);
        return;
    }
    Pun(n-1);
    Point(n);
    P(n-2)
}

void Pun(int n){
    if(n<=1){
        Point(n);
        return;
    }
    Pun(n-1);
    Point(n);
    P(n-2);
}

void Pun(int n){
    if(n<=1){
        Point(n);
        return;
    }
}

void Pun(int n){
    if(n<=1){
        Point(n);
        return;
    }
}
```

Condition satisfied

Condition satisfied

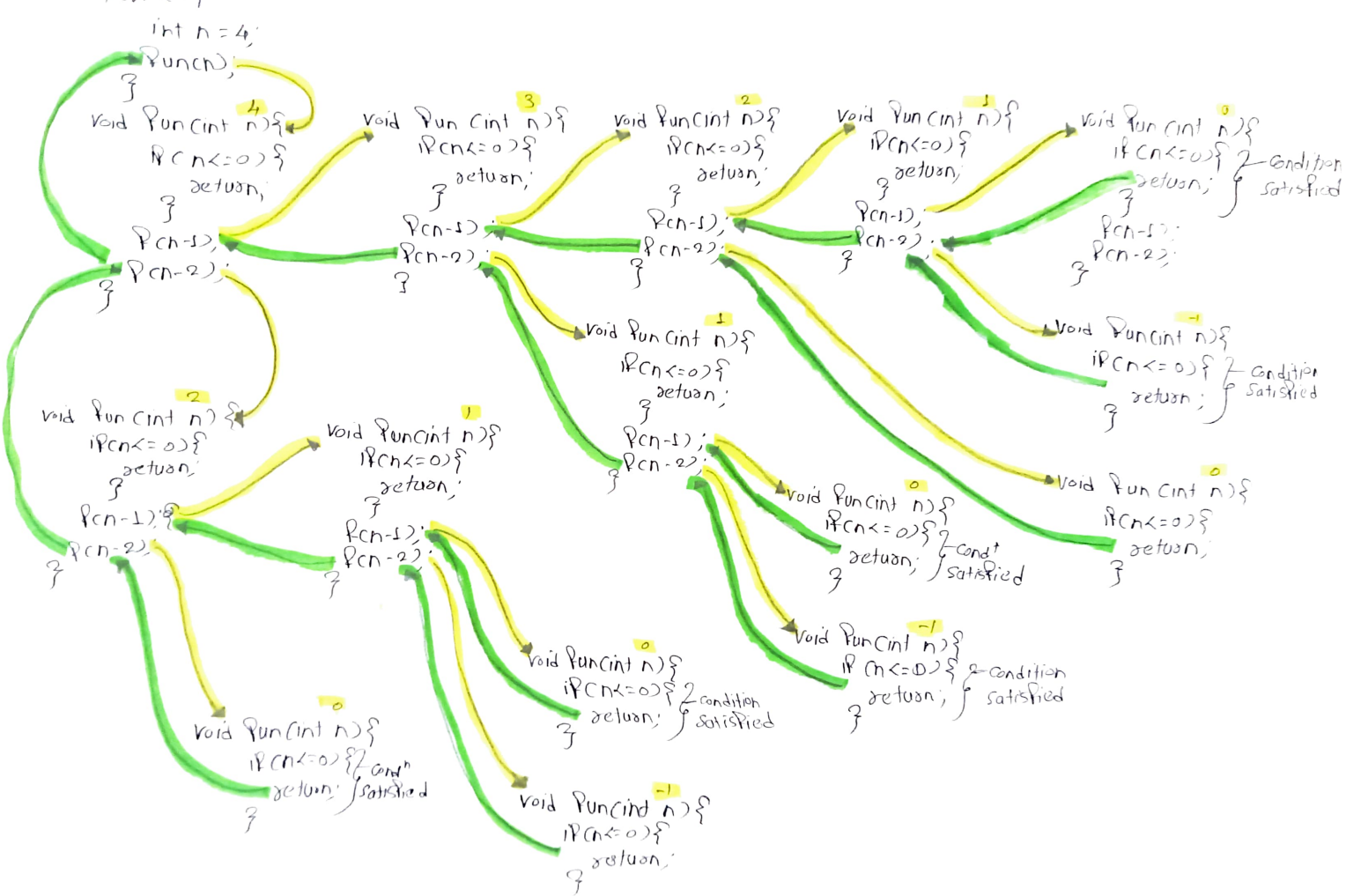
output

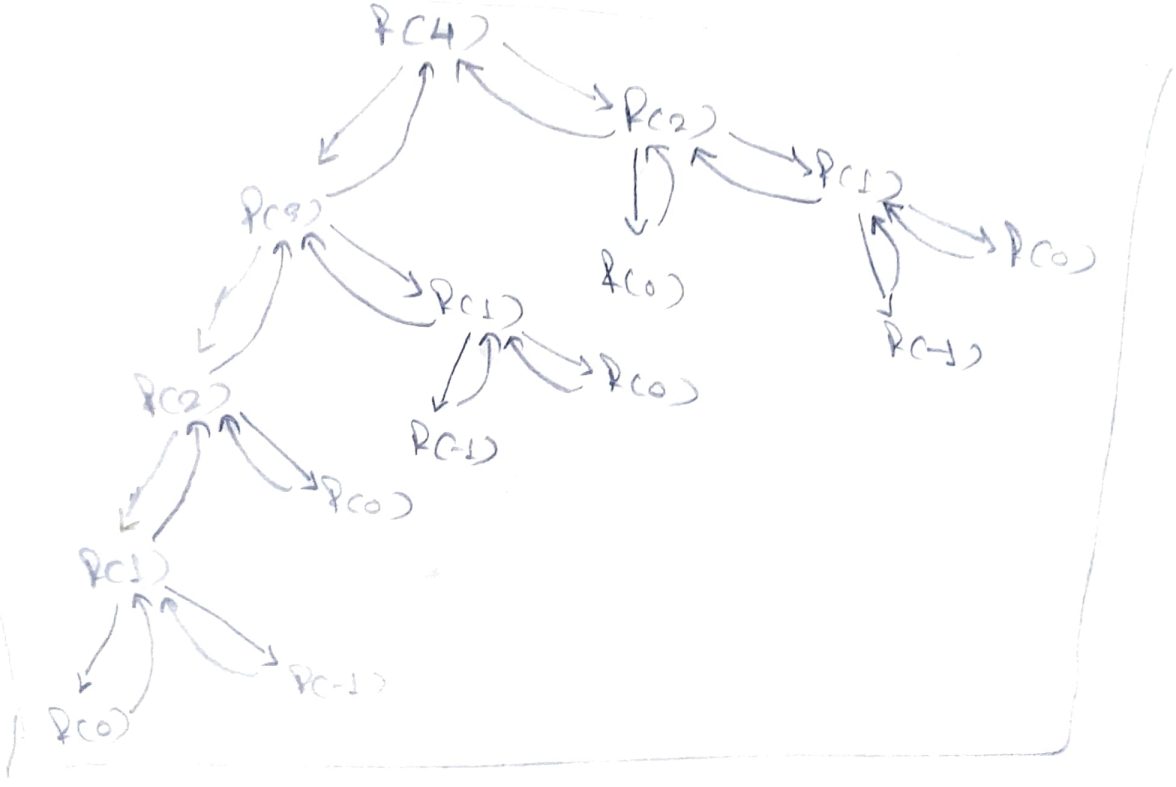
1
2
3
3
1



Total Function call = 5

Q1=> Count Recursive calls:





Total Function call = 15